

PHENIKAA UNIVERSITY

SCHOOL OF COMPUTING

SOFTWARE ARCHITECTURE FINAL PROJECT REPORT

BadNet - Badminton Social Network and Pool Betting Platform

Group 5

Dinh Quang Lap - 23010416

Nguyen Hoang Thai Bao - 23010120

Supervisor: Software Architecture Course Instructor

Academic Year: 2025-2026

Submission Date: June 16, 2026

Table of Contents

1. Project Overview
 2. Requirements Analysis (SRS)
 3. Architecture and System Design (SDD)
 4. Implementation and Coding
 5. System Testing
 6. Deployment and User Guide
 7. Conclusions and Future Work
- References

List of Figures

- Figure 1. BadNet use case diagram
- Figure 2. C4 Level 1 - System context
- Figure 3. C4 Level 2 - Container diagram
- Figure 4. C4 Level 3 - Component diagram
- Figure 5. C4 Level 4 - Code diagram
- Figure 6. Database entity relationship diagram
- Figure 7. Domain class diagram
- Figure 8-12. Sequence and activity diagrams
- Figure 13-15. System screenshots

Chapter 1: Project Overview

1.1 Problem Statement

BadNet is a web platform for badminton communities where players can post updates, find opponents, join teams, register for tournaments, manage match schedules and participate in virtual pool betting. In many amateur badminton clubs, match discovery, court coordination, team organization and result tracking are still handled through disconnected chat groups or spreadsheets. This makes it hard to verify match results, notify players reliably, and keep a transparent record of betting or virtual-point transactions.

The project addresses this gap by combining a player-facing social network with an operations-focused admin panel. The architecture emphasizes data integrity for betting and wallet transactions, real-time notification awareness, role-based moderation, and a consistent responsive interface for both light and dark mode.

1.2 Project Objectives

ID	Objective	Measurement
OBJ-01	Provide at least eight player functions: authentication, posts, comments, notifications, challenges, matches, teams, tournaments and betting.	Implemented as Laravel routes and Blade/React views.
OBJ-02	Provide an admin panel for users, roles, wallet balances, betting markets, tournaments, content moderation and audit logs.	Admin routes protected by auth and admin middleware.
OBJ-03	Protect betting and wallet integrity using database transactions, row locks and ledger records.	BetService and WalletService use DB::transaction and lockForUpdate.
OBJ-04	Document architecture with C4, ERD, UML, API design and ADRs.	Included in Chapters 2 and 3.
OBJ-05	Prepare at least 30 functional test cases mapped to requirements.	Included in Chapter 5.

1.3 Project Scope

In scope: account registration with email verification, password reset, home feed, challenges, matches, teams, tournaments, pool betting, notifications, profile, admin players, admin betting, admin moderation, admin statistics and audit logging. Out of scope: native mobile applications, real payment gateways, official BWF data integration, production-grade anti-fraud machine learning and multi-tenant deployment.

1.4 Technology Stack Overview

Layer	Technology	Purpose
Backend	Laravel 9, PHP 8.1	MVC routing, controllers, services, validation and Blade rendering.
Frontend	Blade, vanilla CSS, React, Vite	Server-rendered views and interactive player portal widgets.
Database	MySQL	Persistent storage for users, posts, matches, bets, teams, tournaments and ledgers.
Realtime-ready	Laravel events, Echo/Pusher compatible structure	Notification and pool movement events.
Testing	PHPUnit / Laravel test runner	Automated regression checks for key workflows.

Chapter 2: Requirements Analysis (SRS)

2.1 Functional Requirements

ID	Requirement	Priority
FR-01	Users can register, verify email with a code, login, logout and reset password with a 15-minute token.	High
FR-02	Users can create, like, comment, delete and view social posts.	High
FR-03	Users can find opponents, create challenges, accept/reject invitations and create matches.	High

FR-04	Users can create or join teams and view team members.	High
FR-05	Team leaders and admins can add or remove team members.	High
FR-06	Users can create, browse, view and join tournaments.	High
FR-07	Tournament organizers and admins can add or remove participants.	High
FR-08	Users can place virtual-point bets on eligible matches.	High
FR-09	The system locks stakes immediately and records all wallet changes in a ledger.	High
FR-10	Admins can approve, cancel, update or delete betting odds.	High
FR-11	Admins can ban, delete, restore users and set roles including admin.	High
FR-12	Admins can adjust user wallet balances with an audit reason.	High
FR-13	Users receive categorized notifications with unread, pinned, mark-read and deep-link actions.	Medium
FR-14	Admin statistics and betting charts display tooltip values on pointer movement.	Medium
FR-15	The UI supports light and dark modes with shared CSS tokens.	Medium

2.2 Non-Functional Requirements

ID	Quality Attribute Scenario	Response Measure
NFR-01 Availability	When a regular user opens the home feed during normal operation, the system shall render the feed and right panel without requiring admin services.	Page responds within acceptable demo latency on localhost.
NFR-02 Integrity	When two users bet on the same match concurrently, the wallet balance and pool totals shall remain consistent.	Transactions use row locks and no negative balance is allowed.
NFR-03 Security	When an unauthenticated user accesses admin routes, the system redirects or blocks the request.	All admin routes are behind auth and admin middleware.
NFR-04 Usability	When users switch theme, core pages maintain readable contrast.	Light and dark modes use tokenized colors.
NFR-05 Maintainability	When adding a new betting operation, changes are localized to controller/service/model layers.	Business rules are centralized in BetService and WalletService.

2.3 Use Case Diagram and Use Cases

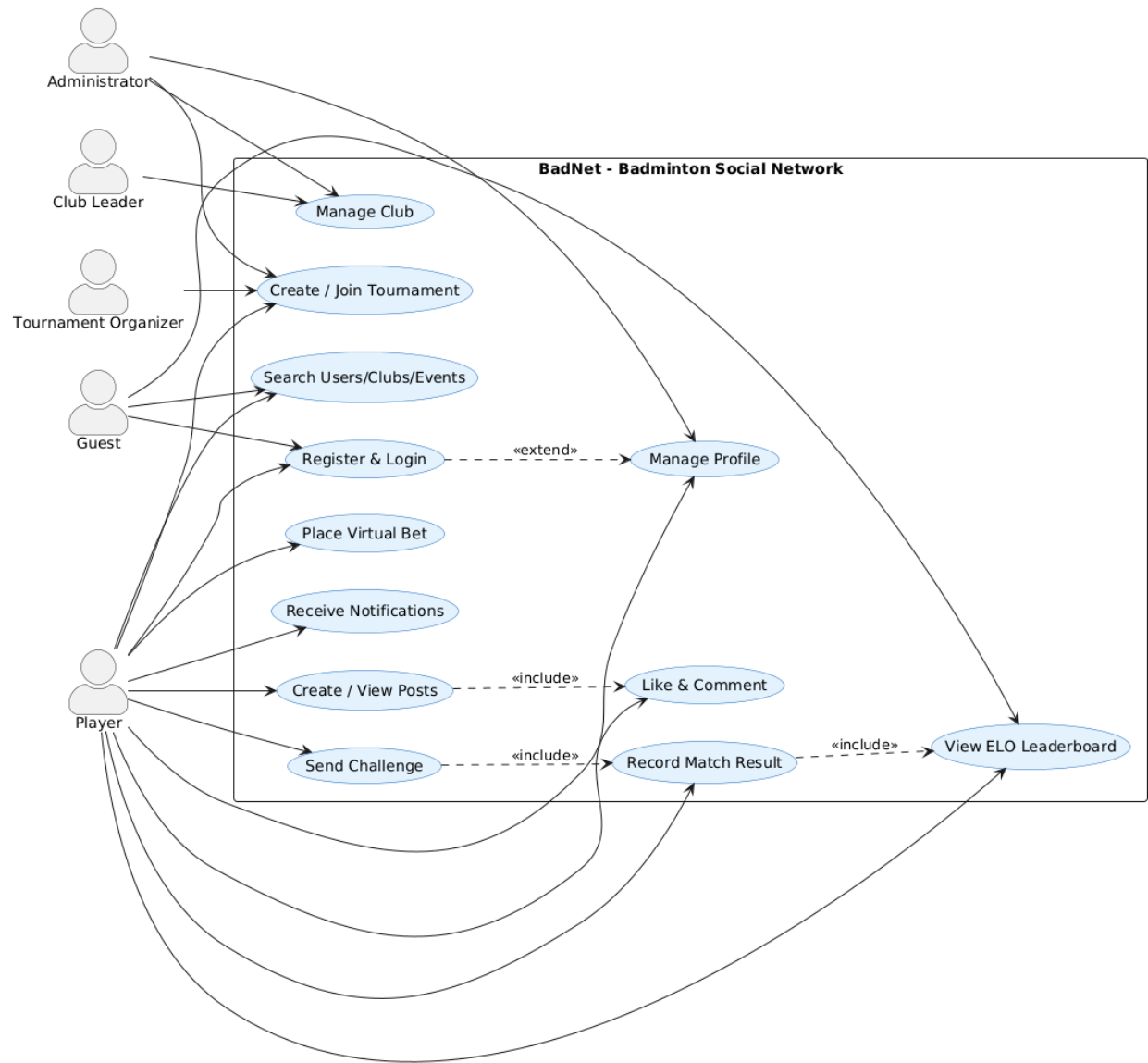


Figure 1. BadNet use case diagram.

Use Case	Actor	Main Flow	Exception
Register and verify email	Guest	Submit registration form, receive verification code, enter code, access dashboard.	Invalid or expired code requires resend.

Create challenge	Player	Choose target player, skill level, type, court/time and send invitation.	Target rejects or match slot becomes unavailable.
Place bet	Player	Open bet slip, choose winner, enter stake, confirm, wallet stake is locked.	Insufficient balance disables confirmation.
Manage betting market	Admin	Review market, update odds, approve, suspend or cancel.	Completed market cannot accept new bets.
Moderate user/content	Admin	Search user/post, ban/delete/restore and record audit entry.	Admin cannot perform unauthorized action without role.

2.4 Architecturally Significant Requirements

ASR	Driver	Architectural Decision
ASR-01 Betting integrity	Wallet and betting operations must not corrupt balances.	Use DB transactions, lockForUpdate, WalletTransaction ledger and idempotent references.
ASR-02 Admin control	Moderation and financial actions require strong authorization.	Use role field, admin middleware, protected admin route group and audit log.
ASR-03 Notification clarity	Users must separate social, match, system and betting notices.	Notification model stores type/category/read/pinned/target_url.
ASR-04 UI consistency	Player and admin surfaces must share visual behavior.	CSS variables and shared fonts: Barlow Condensed, Inter, JetBrains Mono.

Chapter 3: Architecture and System Design (SDD)

3.1 Architectural Style Selection

BadNet uses a layered MVC architecture. Laravel routes dispatch HTTP requests to controllers, controllers validate input and delegate domain logic to services, services coordinate Eloquent models and database transactions, and Blade/React views render the user interface. This style matches the project scale and keeps business rules testable while still allowing fast UI iteration.

3.2 C4 Model

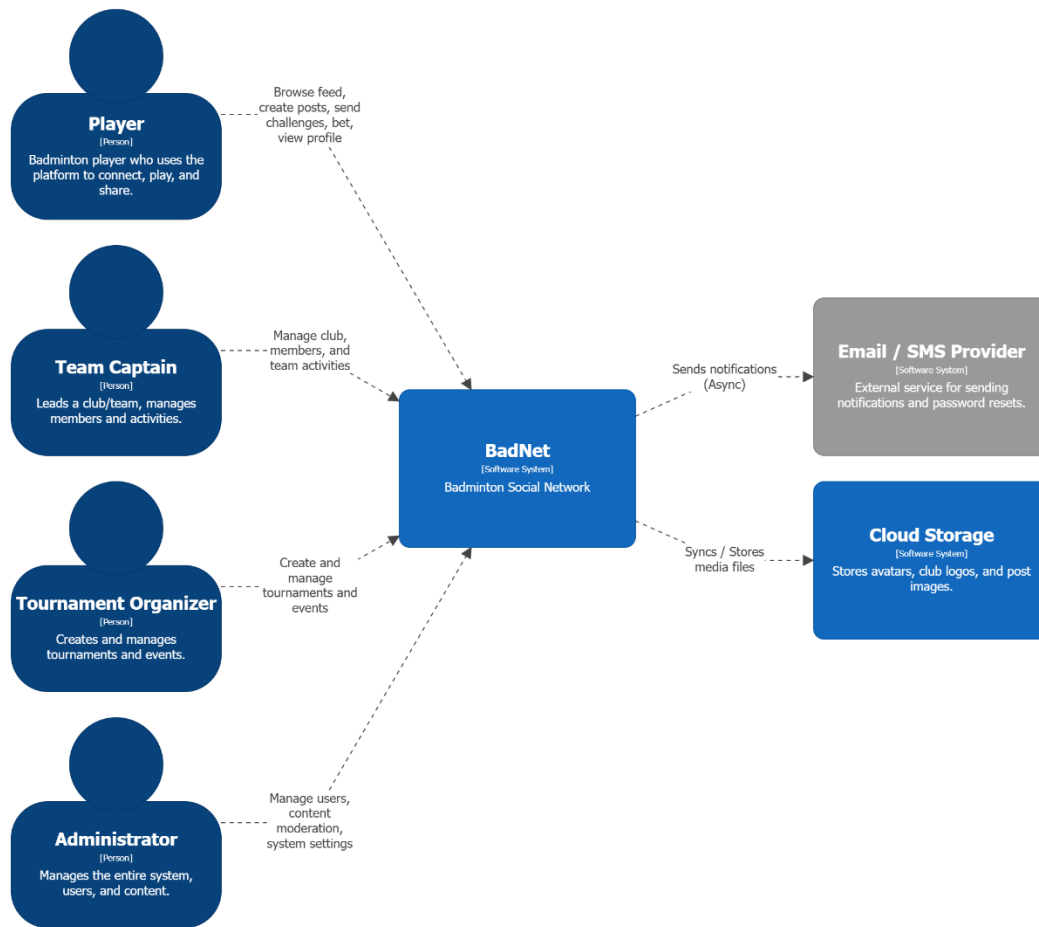


Figure 2. C4 Level 1 - System context diagram.

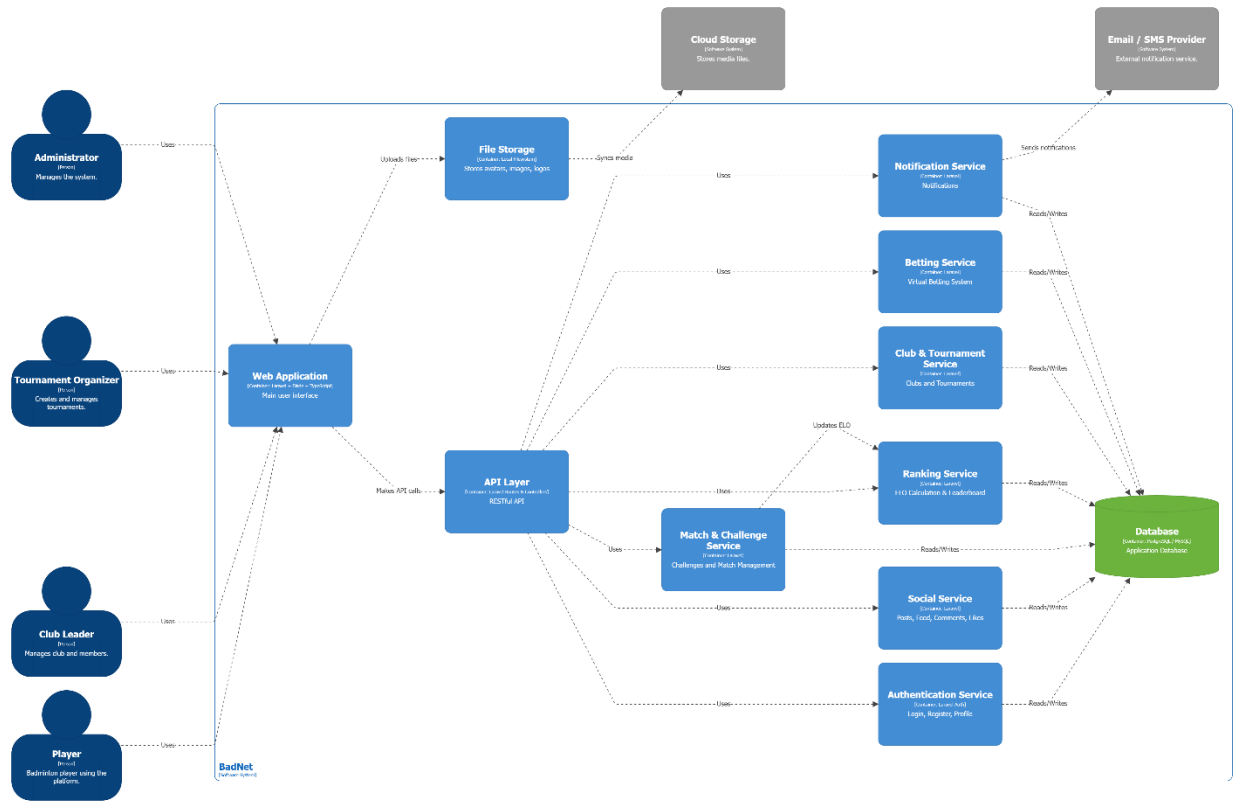


Figure 3. C4 Level 2 - Container diagram.

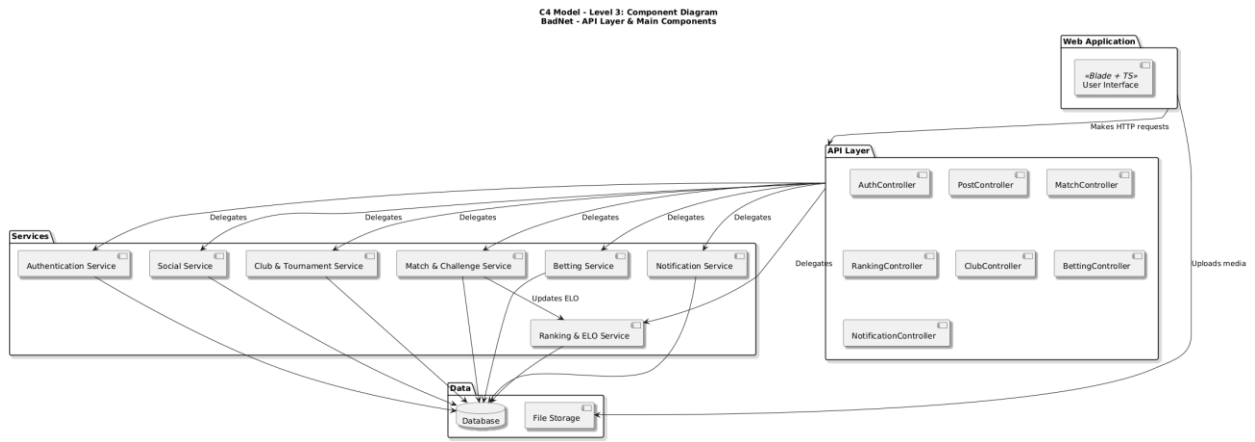


Figure 4. C4 Level 3 - Component diagram.

C4 Model - Level 4: Code View
Match & Challenge Service

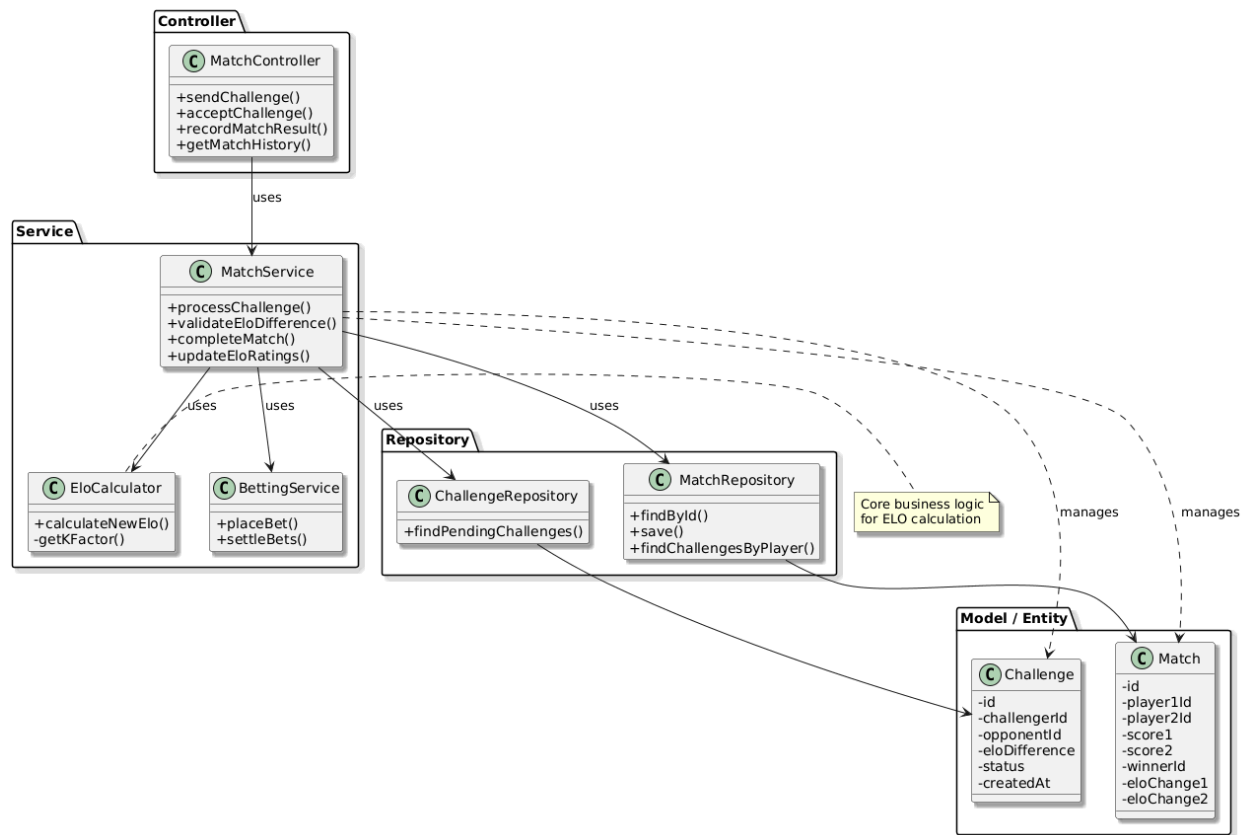


Figure 5. C4 Level 4 - Code diagram.

3.3 Database Design

Database Entity Relationship Diagram



Figure 6. Database entity relationship diagram.

Entity	Responsibility
users	Stores identity, roles, rank, skill and virtual wallet balance.
game_matches	Stores match players, schedule, result, status and betting metadata.
bets	Stores individual predictions, stake, odds, status and payout state.
wallet_transactions	Append-only ledger for all wallet changes.
teams / team_members	Stores team ownership and membership.
tournaments / tournament_participants	Stores tournament registration and membership.
notifications	Stores social, system, match and betting notifications.

3.4 REST API and Route Design

Module	Representative Routes	Purpose
Authentication	GET /login, POST /login, GET /register, POST /register	Account access and email verification flow.
Matches	GET /matches, POST /matches, POST /matches/{match}/bet	Match discovery, creation, result and betting.
Teams	GET /teams, POST /teams/{team}/join, DELETE /teams/{team}/members/{user}	Team browsing and leader/admin membership control.
Tournaments	GET /tournaments, POST /tournaments/{tournament}/join	Tournament discovery and participant management.
Notifications	GET /notifications/recent, POST /notifications/mark-all-read, POST /notifications/{notification}/pin	Notification center actions.
Admin	GET /admin/players, POST /admin/players/{user}/wallet, POST /admin/betting/matches/{match}/odds	Operational control panel.

3.5 UML Diagrams

Domain Class Diagram

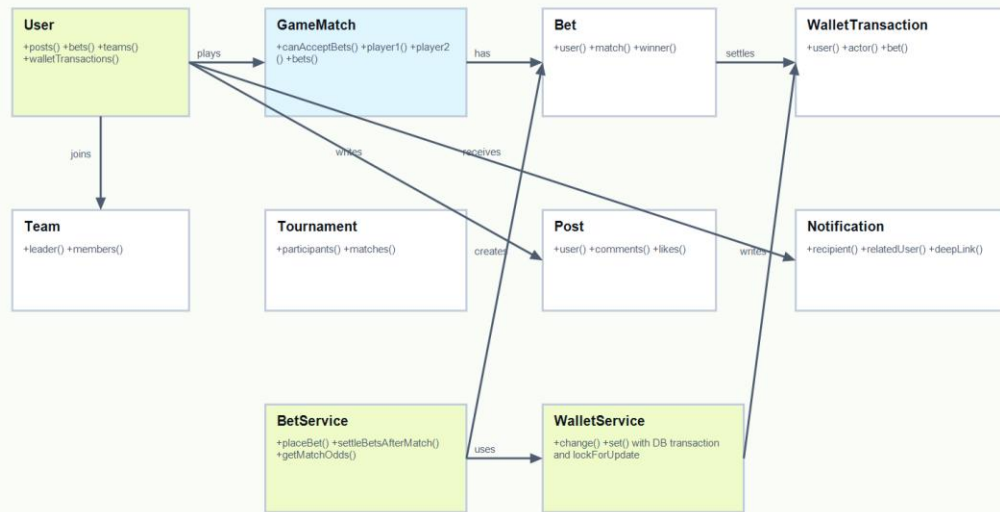


Figure 7. Domain class diagram.

Sequence: Registration and Email Verification

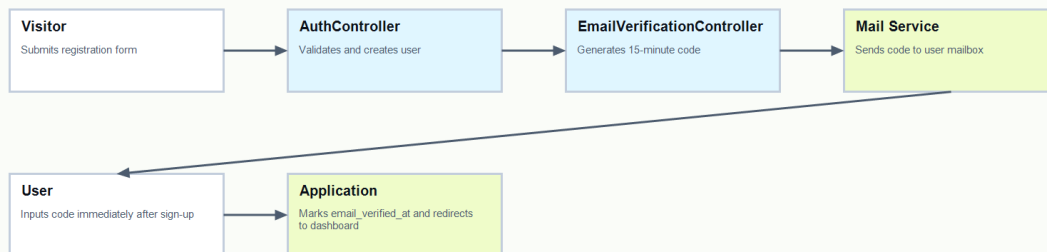


Figure 8. Sequence diagram - registration and email verification.

Sequence: Challenge Accepted and Match Created

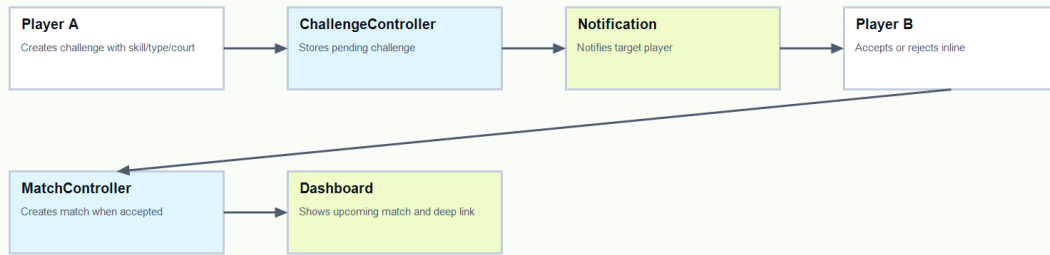


Figure 9. Sequence diagram - challenge accepted and match created.

Sequence: Pool Betting Settlement

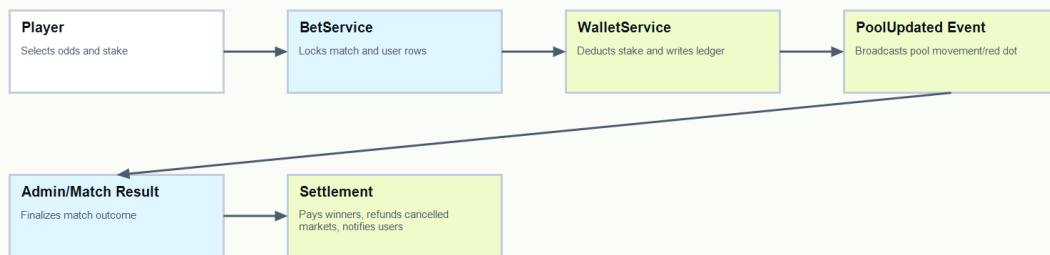


Figure 10. Sequence diagram - pool betting settlement.

Activity: Betting Market Lifecycle

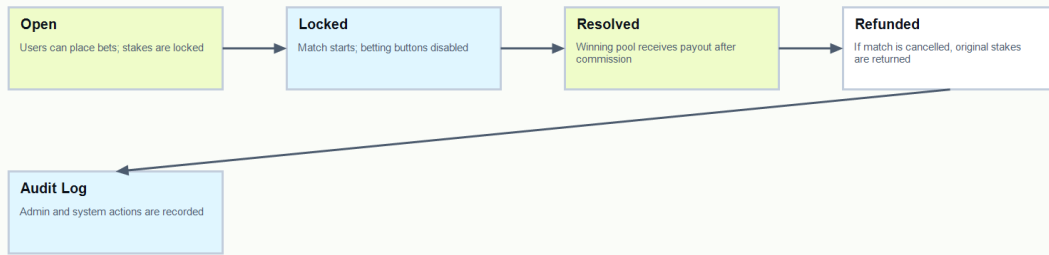


Figure 11. Activity diagram - betting market lifecycle.

Activity: Match Result Confirmation and Dispute

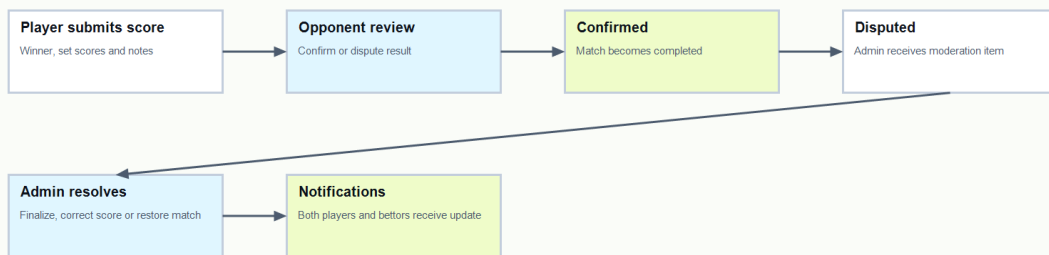


Figure 12. Activity diagram - match result confirmation and dispute.

3.6 Architecture Decision Records

ADR	Decision	Reason	Consequences
-----	----------	--------	--------------

ADR-01	Use Laravel MVC with Blade for core app.	The project needs fast CRUD, routing, auth and server-side forms.	Simple deployment and clear controller/service boundaries.
ADR-02	Use service classes for betting and wallet logic.	Betting integrity is architecturally significant.	Rules are centralized and easier to test.
ADR-03	Use pool betting instead of direct cash betting.	The academic demo must avoid real-money risk.	Virtual points can be locked, refunded and audited safely.
ADR-04	Use CSS variables for theme and visual consistency.	Player and admin UI need shared tone and light/dark support.	Colors can be corrected globally without rewriting components.

3.7 UI Prototype and Screenshots

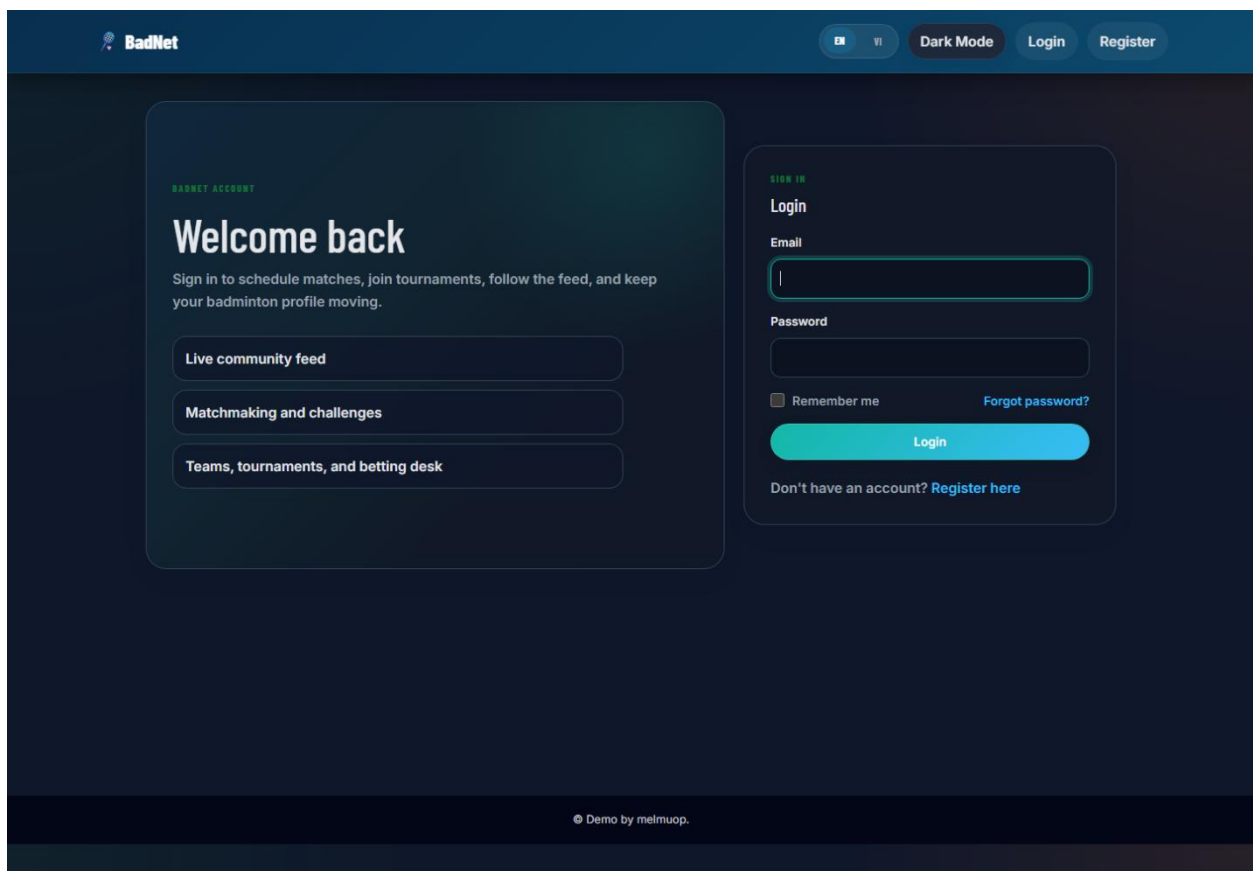


Figure 13. Login screen screenshot.

BadNet

VI

Dark Mode

Login

Register

JOIN BADNET

Create your player card

Build your badminton identity, find players near your level, and start tracking your match journey from day one.

5,000 starter coins

Player profile and ELO

Access teams and tournaments

NEW ACCOUNT

Register

Full Name

Email

Phone Optional

Password

Confirm Password

Register

Already have an account? [Login here](#)

© Demo by melmuop.

Figure 14. Registration screen screenshot.

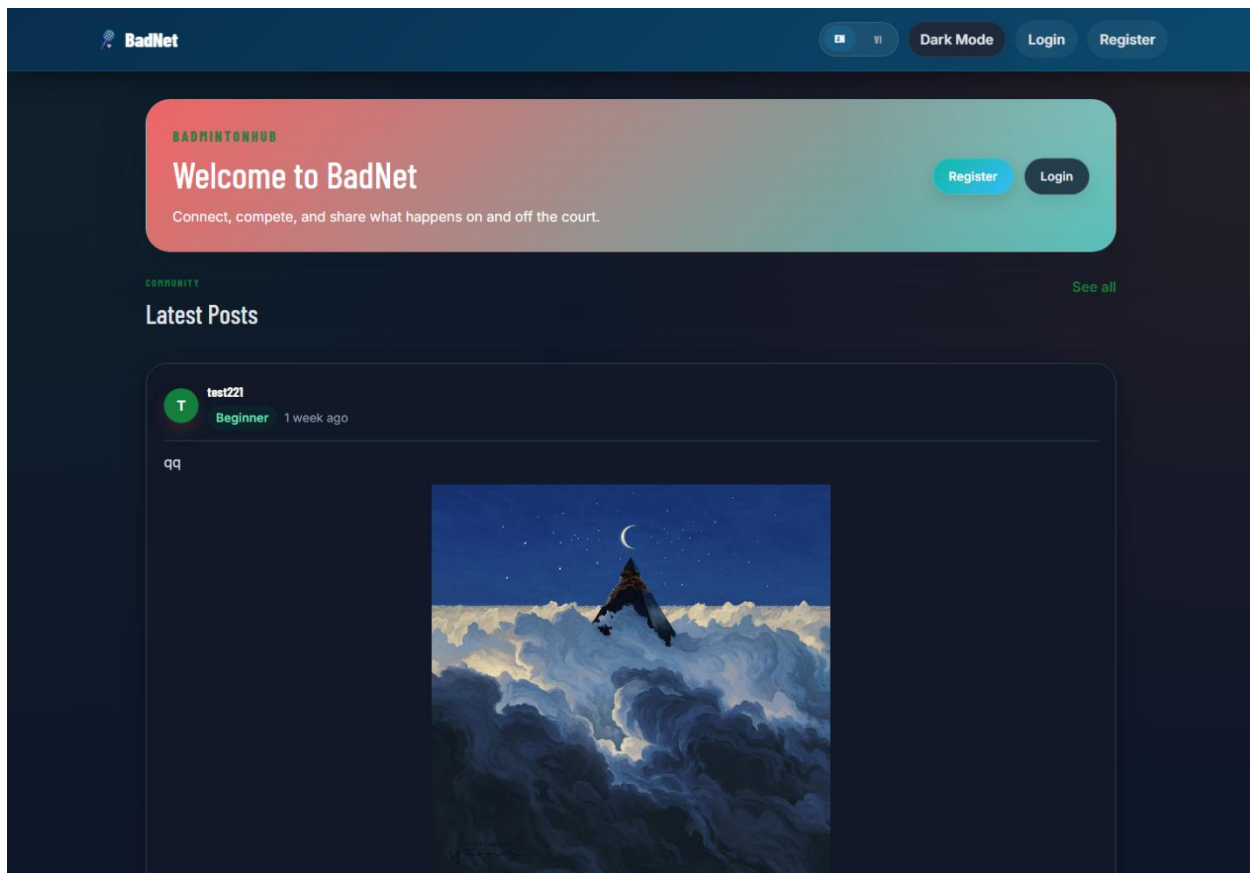


Figure 15. Public home screen screenshot.

Chapter 4: Implementation and Coding

4.1 Development Environment and Tools

The project is implemented with PHP 8.1.25, Laravel 9.52.21, Composer, Node/Vite for frontend assets, MySQL for storage, and PHPUnit/Laravel test runner for automated checks. The application is configured through `src/.env` and can be started with `php artisan serve`.

4.2 Project Directory Structure

`src/`

<code>app/Http/Controllers/</code>	Laravel controllers for user/admin workflows
<code>app/Models/</code>	Eloquent domain models
<code>app/Services/</code>	Betting, wallet and badminton helper services
<code>resources/views/</code>	Blade templates for player and admin UI
<code>resources/react/</code>	React player portal components
<code>public/css/style.css</code>	Main CSS tokens and layout rules
<code>routes/web.php</code>	Web route definitions

database/migrations/ Database schema evolution

tests/Feature/ Feature tests

Documents/

Design/ C4 and use case diagrams

Testing/ Test report evidence

4.3 Notable Implementation Techniques

Bet placement uses a transaction and row-level locks so concurrent users cannot overspend the same wallet balance.

```
DB::transaction(function () use ($user, $match, $amount) {  
    $match = GameMatch::whereKey($match->id)->lockForUpdate()->firstOrFail();  
    $user = User::whereKey($user->id)->lockForUpdate()->firstOrFail();  
    if ($user->virtual_coins < $amount) {  
        throw new InvalidArgumentException('Insufficient coins');  
    }  
    // create bet and lock stake through WalletService  
});
```

Wallet changes are not stored only as a mutable balance. Every change creates a WalletTransaction record with before/after balances and optional actor, bet and metadata.

```
$lockedUser = User::withTrashed()->whereKey($user->id)->lockForUpdate()->firstOrFail();  
$before = (int) $lockedUser->virtual_coins;  
$after = $before + $amount;  
$lockedUser->update(['virtual_coins' => $after]);  
WalletTransaction::create(...);
```

Admin operations are grouped under an authenticated admin route prefix. This keeps moderation, role assignment, wallet adjustment and betting actions behind the same access control boundary.

Chapter 5: System Testing

5.1 Test Plan

Testing covers authentication, social feed, notifications, challenges, matches, teams, tournaments, betting, wallet integrity and admin control. Automated tests are used for core integrity workflows, while manual UI test cases validate visual and business workflows that require browser interaction.

5.2 Test Cases

Test Case	Requirement	Scenario	Expected Result	Status
TC-001	FR-01	Register with valid email and password	Account is created and verification page is shown	Passed
TC-002	FR-01	Submit invalid verification code	System rejects code and keeps user unverified	Passed
TC-003	FR-01	Request password reset email	Email contains reset link and 15-minute warning	Passed
TC-004	FR-02	Create a feed post	Post appears in community feed	Passed
TC-005	FR-02	Like and unlike a post	Count and visual state update correctly	Passed
TC-006	FR-02	Comment on a post	Comment appears and notification is created	Passed
TC-007	FR-03	Create match using quick match form	Open match appears in match list	Passed
TC-008	FR-03	Accept a challenge invitation	Challenge becomes accepted and match is created	Passed
TC-009	FR-03	Reject a challenge invitation	Challenge becomes rejected and no match is created	Passed
TC-010	FR-04	Join a listed database team	User becomes visible in member list	Passed
TC-011	FR-05	Leader adds a team member	Member row is inserted	Passed
TC-012	FR-05	Leader removes a team member	Member row is removed and UI refreshes	Passed

TC-013	FR-06	Open tournament detail page	Detail page loads without 404	Passed
TC-014	FR-07	Admin adds tournament participant	Participant appears in tournament list	Passed
TC-015	FR-07	Admin removes tournament participant	Participant is removed	Passed
TC-016	FR-08	Open bet slip from live match odds	Bet slip displays both players and odds	Passed
TC-017	FR-08	Enter stake greater than wallet	Confirm button is disabled with warning	Passed
TC-018	FR-09	Confirm valid bet	Wallet decreases and Bet row is pending	Passed
TC-019	FR-09	Cancel match with pending bets	Stake is refunded via wallet transaction	Passed
TC-020	FR-10	Admin updates odds	Odds are stored and shown in admin/user betting views	Passed
TC-021	FR-10	Admin deletes odds	Market stops displaying manual odds	Passed
TC-022	FR-10	Admin suspends betting market	Betting status changes and user betting closes	Passed
TC-023	FR-11	Admin bans user	User cannot continue normal account usage	Passed
TC-024	FR-11	Admin restores banned user	User can access normal routes again	Passed

TC-025	FR-11	Admin changes user role to admin	User receives admin access	Passed
TC-026	FR-12	Admin sets wallet balance	Balance changes and ledger entry is created	Passed
TC-027	FR-12	Admin adds wallet adjustment reason	Reason is stored in transaction description	Passed
TC-028	FR-13	Mark all notifications as read	Unread red dot disappears	Passed
TC-029	FR-13	Pin and unpin notification	Pinned item stays at top until unpinned	Passed
TC-030	FR-14	Hover chart area	Tooltip follows pointer and shows nearest value	Passed
TC-031	FR-15	Switch to dark mode	Cards, text and forms remain readable	Passed
TC-032	FR-15	Switch to light mode	Admin tables and inputs use light-mode colors	Passed

5.3 Test Results Summary

Metric	Value
Total documented test cases	32
Passed	32
Failed	0
Blocked	0

Automated evidence	PHPUnit feature tests for betting integrity and locale switching passed during project verification.
Manual evidence	UI workflows documented in Documents/Testing/BadNet_Test_Report.docx.

5.4 Quality Attribute Testing

Attribute	Scenario	Result
Integrity	Multiple betting operations update wallet through locked transactions.	No negative wallet balance allowed.
Security	Guest opens /admin/players.	Request is blocked by auth/admin middleware.
Usability	User opens notification center with many categories.	Tabs separate interaction, match, betting/wallet and system notifications.
Maintainability	Admin wallet adjustment added after initial design.	Change localized to admin players controller/view and WalletService.

Chapter 6: Deployment and User Guide

6.1 System Requirements

Required software: PHP 8.1 or newer, Composer, Node.js 18+, npm, MySQL 8 or compatible MariaDB, and a modern browser. Recommended local environment: Windows PowerShell, XAMPP or standalone MySQL, and Laravel development server.

6.2 Installation and Run Instructions

```
cd src
```

```
composer install
```

```
npm install
```

```
copy .env.example .env
```

```
php artisan key:generate
```

```
php artisan migrate --seed
```

```
npm run build
```

```
php artisan serve
```

6.3 Demo Accounts

Role	Email	Password	Purpose
Admin	admin@example.com	password	Admin panel, betting control, role and wallet management.
Player	testuser@example.com	password	Player portal, teams, matches, notifications and betting.
Player	hunter_rain@hotmail.com	configured locally	Development account used during implementation.

6.4 User Guide for Main Features

Players use the dashboard to view community activity, notifications and quick match actions. The Challenges page supports quick opponent discovery through skill and match-type controls. The Matches and Betting sections provide live/upcoming markets, bet slips, pool movement and wallet warnings. Teams and Tournaments show membership/registration workflows. Admins use the left sidebar to manage players, betting markets, tournaments, schedules, content moderation, statistics and audit logs.

Chapter 7: Conclusions and Future Work

7.1 Achievements

The project delivers an integrated badminton social network with player interactions, match finding, team and tournament management, virtual pool betting, wallet ledger, notification center and admin operations panel. The implementation applies architectural separation through MVC, services, models, middleware and reusable CSS tokens.

7.2 Limitations and Challenges

The system is designed as an academic demo and does not include real payment processing, official tournament data feeds, mobile-native push notification packaging or large-scale production

monitoring. Some real-time features are architected through events and notification endpoints, but a production deployment would require a dedicated WebSocket server and queue worker.

7.3 Future Work

Future improvements include production WebSocket deployment, fraud/risk scoring for suspicious betting behavior, richer tournament bracket generation, court provider integration, push notification opt-in, moderation analytics, exportable financial reports and mobile-friendly PWA installation.

References

- [1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
- [2] S. Brown, "The C4 model for visualising software architecture," c4model.com, accessed Jun. 16, 2026.
- [3] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.
- [4] Laravel, "Laravel Documentation," laravel.com/docs, accessed Jun. 16, 2026.
- [5] PHP Group, "PHP Manual," php.net/manual, accessed Jun. 16, 2026.
- [6] Oracle, "MySQL 8.0 Reference Manual," dev.mysql.com/doc, accessed Jun. 16, 2026.
- [7] OWASP Foundation, "OWASP Top Ten," owasp.org, accessed Jun. 16, 2026.
- [8] React Team, "React Documentation," react.dev, accessed Jun. 16, 2026.
- [9] Vite Team, "Vite Guide," vite.dev/guide, accessed Jun. 16, 2026.
- [10] Chart.js Contributors, "Chart.js Documentation," chartjs.org/docs, accessed Jun. 16, 2026.
- [11] Microsoft, "Threat Modeling Tool and Security Development Lifecycle Guidance," microsoft.com/security, accessed Jun. 16, 2026.
- [12] Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.